



Resource Management for Distributed Binary Neural Networks in Programmable Data Plane

Fatemeh Babaei*, Mahdi Dolati*, Mojtaba Mozhganfar†, Sina Darabi^, **Farzad Tashtarian**#

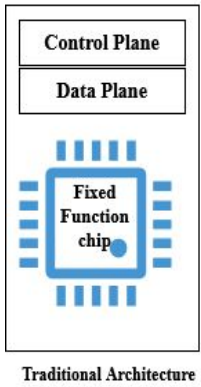
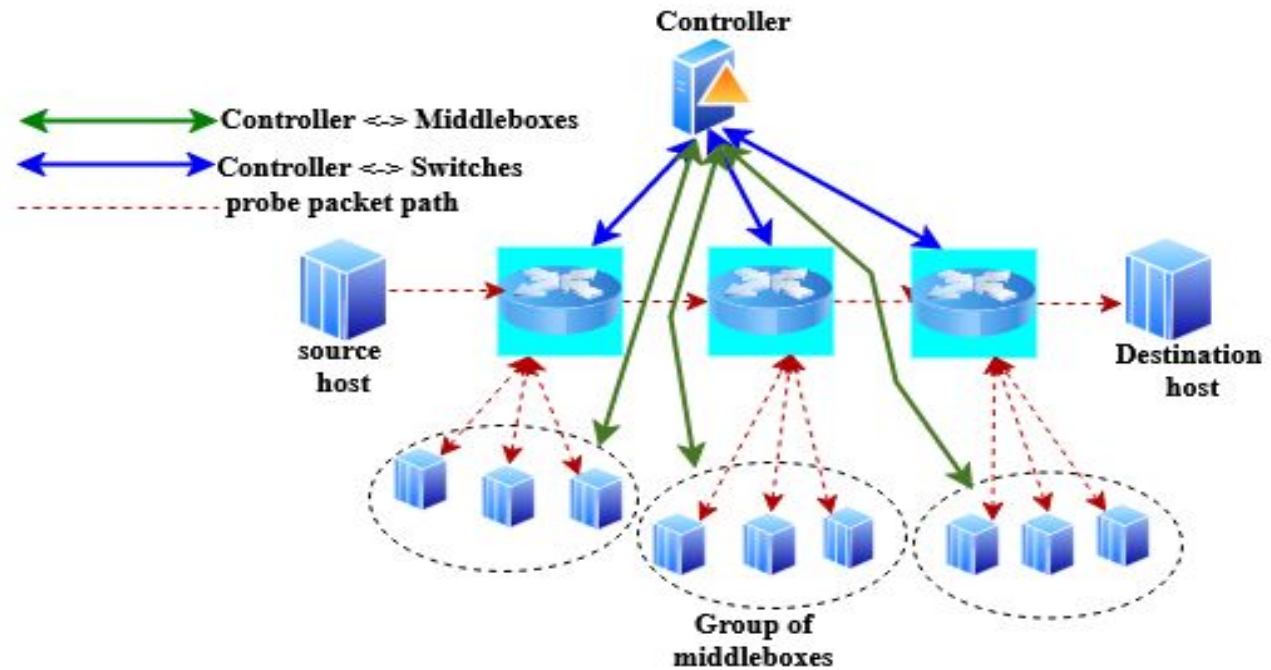
*Department of Computer Engineering, Sharif University of Technology, Tehran, Iran

†School of Electrical and Computer Engineering, College of Engineering, University of Tehran, Tehran, Iran

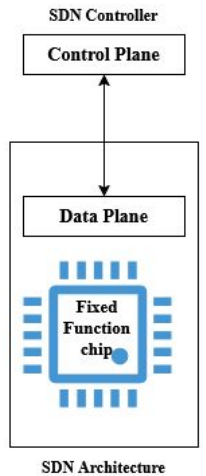
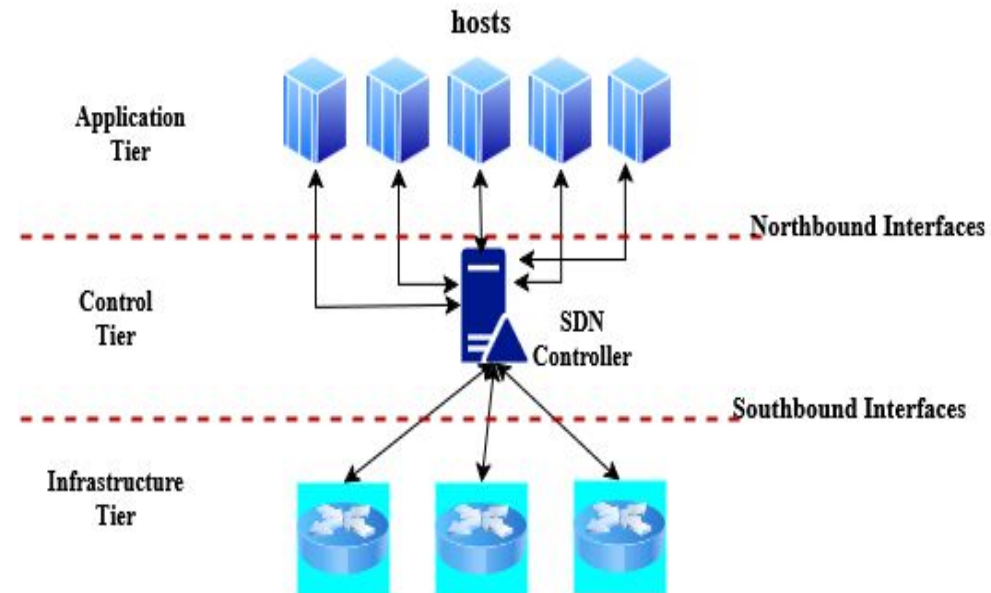
^Università della Svizzera italiana (USI), Lugano, Switzerland

#Christian Doppler Laboratory ATHENA, Alpen-Adria-Universität Klagenfurt, Austria

- Traditional Network
 - Middleboxes

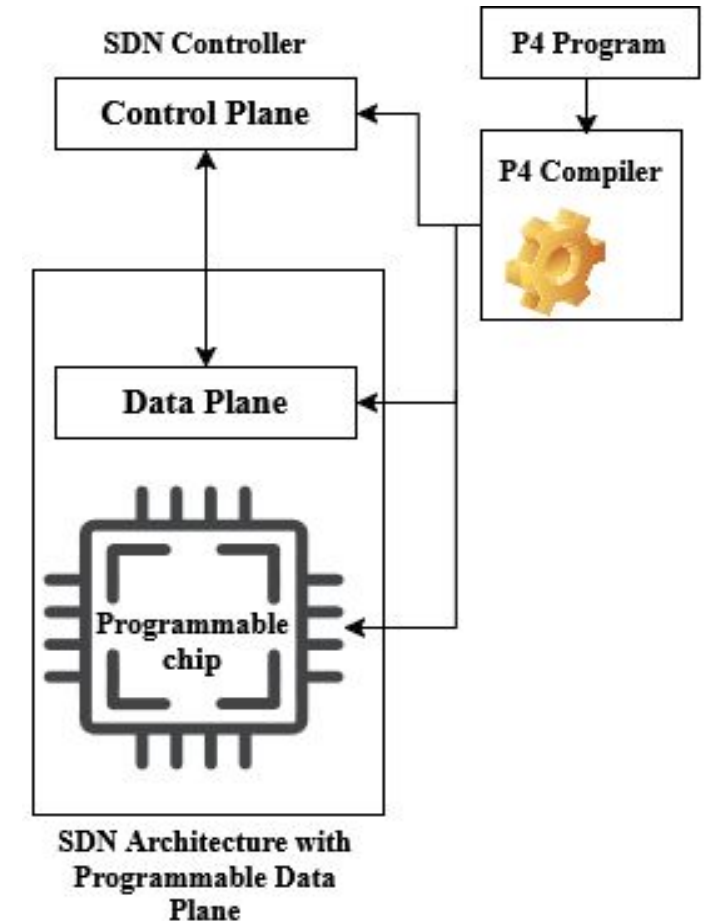


- Software-Defined Network (SDN)
 - Separating the control panel and data plane



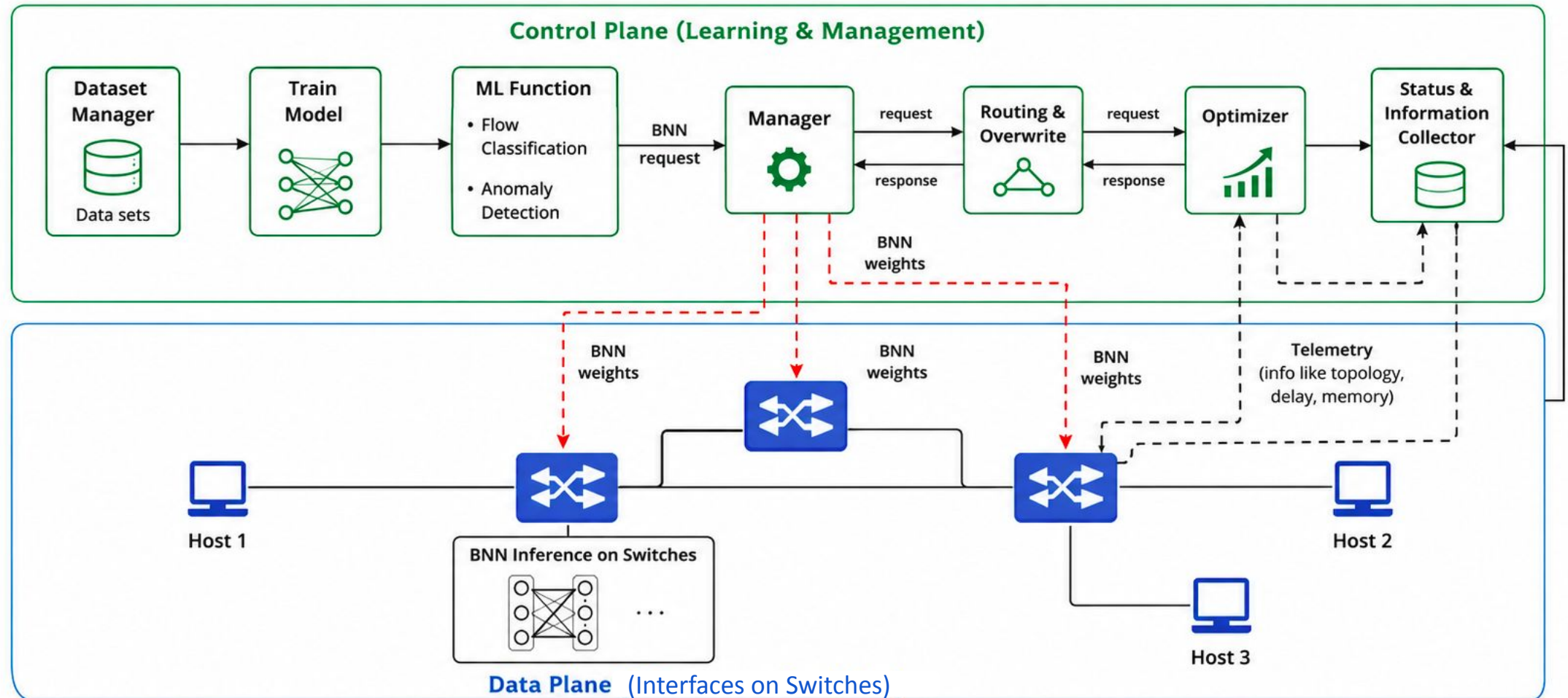
- Programmable Network

- Enable **line-rate in-network computation** through programmable data planes
- Support **low-latency distributed ML inference**
 - **BNNs** improve efficiency under hardware constraints
 - Binary Neural Networks (BNNs) are attractive in computer networking because they drastically reduce memory usage, computation cost, and energy consumption.
- Challenges
 - Existing approaches lack **network-wide resource optimization**
- Solution
 - Optimized distributed BNN placement



System model

1. Function learning in the control plane
2. Distribution of learned weights
3. Inference execution on switches



Problem Formulation

- **Input**

- set of switches $\mathcal{S}$
 - each switch $s \in \mathcal{S}$
 - Packet Header Vector (PHV) capacity C_{PHV}^s
 - SRAM capacity C_{SRAM}^s
 - ALU capacity C_{ALU}^s
 - distance between any pair of switches $d(s, s')$
- set of BNNs $\mathcal{B}$
 - Each BNN $b \in \mathcal{B}$
 - set of layers $\mathcal{L}$
 - Each layer $\ell \in \mathcal{L}$ consumes a certain amount of PHV, SRAM, and ALU denoted by $\text{PHV}(\ell)$, $\text{SRAM}(\ell)$, and $\text{ALU}(\ell)$
 - Relative popularity compared to other BNNs p_b

- **Output**

- Binary Decision variables $X_{\ell, s}$ that indicate whether layer ℓ is placed on switch s

Problem Formulation

- **Objective**

1. The resources must be utilized
2. The popularity of BNNs must affect the resource allocation
3. The delay incurred due to the split of BNNs must be minimized

$$\max \gamma = \sum_{\ell \in \mathcal{L}} n_{\ell} - \tau_1 \sum_{\ell \in \mathcal{L}} \left| \frac{n_{\ell}}{\hat{N}} - \hat{p}_{\ell} \right| - \tau_2 \sum_{b \in \mathcal{B}} \sum_{(\ell, \ell') \in b} d \left(\sum_{s \in \mathcal{S}} X_{\ell, s} \cdot s, \sum_{s' \in \mathcal{S}} X_{\ell', s'} \cdot s' \right)$$

Adjustable coefficients

- $n_{\ell} = \sum_{s \in \mathcal{S}} X_{\ell, s}$: number of layer ℓ is deployed

- $\hat{p}_{\ell} = \frac{\sum_{b \in \mathcal{B}} p_b \cdot \sum_{\ell' \in \mathcal{L}_b} \delta(\ell, \ell')}{\hat{N}}, \hat{N} = \sum_{b \in \mathcal{B}} p_b \cdot |\mathcal{L}_b|, \delta(\ell, \ell') = \begin{cases} 1 & \text{if } \ell = \ell' \\ 0 & \text{otherwise} \end{cases}$

Normalized popularity layer ℓ

Problem Formulation

- Constraints

- The total PHV consumption by the layers should not exceed the PHV capacity of the switch.

$$\sum_{b \in \mathcal{B}} \sum_{\ell \in \mathcal{L}_b} X_{\ell,s} \cdot \text{PHV}(\ell) \leq C_{\text{PHV}}^s, \quad \forall s \in \mathcal{S}$$

- The memory consumed by the layers should not exceed the SRAM capacity of the switch.

$$\sum_{b \in \mathcal{B}} \sum_{\ell \in \mathcal{L}_b} X_{\ell,s} \cdot \text{SRAM}(\ell) \leq C_{\text{SRAM}}^s, \quad \forall s \in \mathcal{S}$$

- The consumption of ALU should not exceed the available capacity.

$$\sum_{b \in \mathcal{B}} \sum_{\ell \in \mathcal{L}_b} X_{\ell,s} \cdot \text{ALU}(\ell) \leq C_{\text{ALU}}^s, \quad \forall s \in \mathcal{S}$$

- each layer is deployed on at least one switch

$$\sum_{s \in \mathcal{S}} X_{\ell,s} \geq 1, \quad \forall \ell \in \mathcal{L}.$$

Proposed solution

- **BNN Placement with Layer Splitting (BPLS)** relaxes binary placement into continuous values to efficiently solve large-scale optimization
- A three-phase post-processing algorithm converts fractional solutions into valid binary layer-to-switch assignments.
 1. Initial Assignment: select the best feasible switch based on delay, popularity, and capacity.
 2. Assignment Expansion: layers are placed on feasible switches if resource limits and delay threshold are satisfied.
 3. Assignment Guarantee: ensures full layer assignment by reusing best switches and freeing capacity when needed.

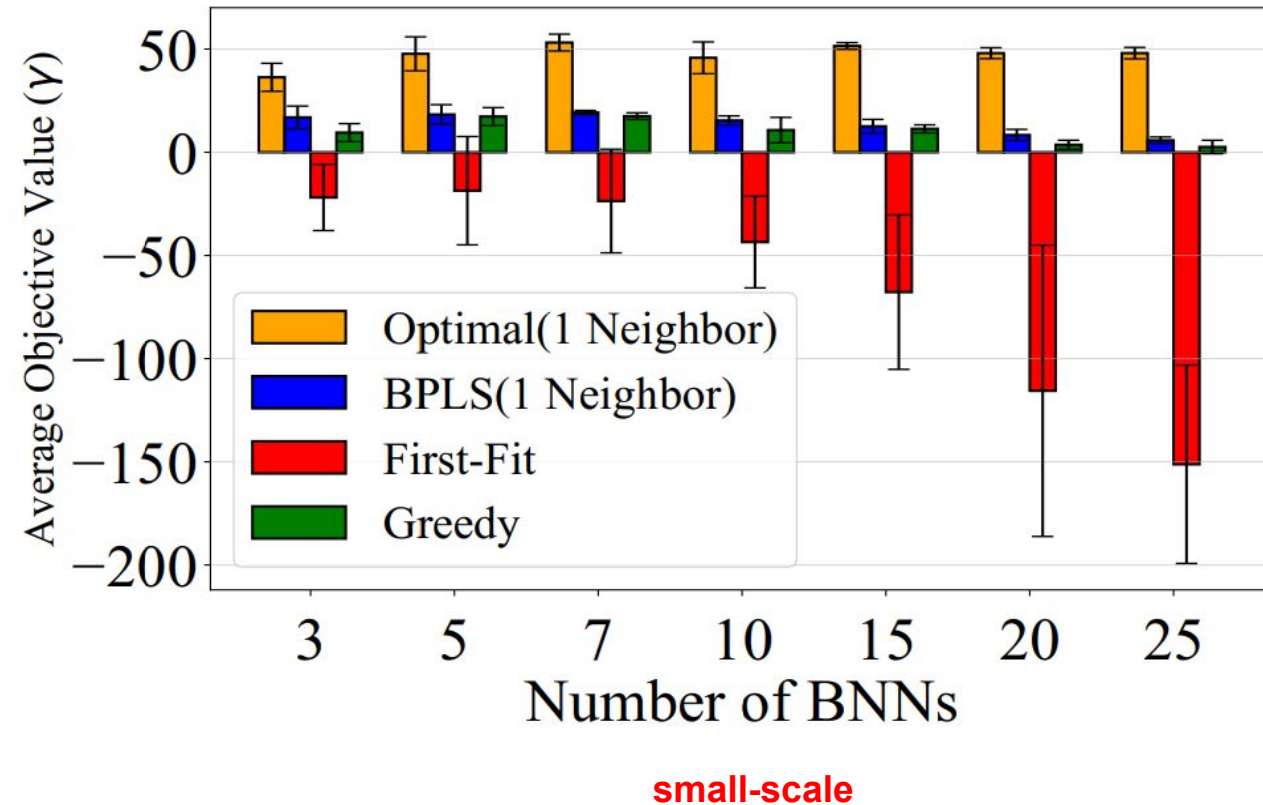
Evaluation

- Experimental setup
 - BPLS is evaluated in a Mininet-based simulation on Linux using a fat-tree topology with bmv2 switches.

Parameter	Value
Network topology	Fat-tree with branching factor 2
No. of switches (small-scale)	7
No. of switches (large-scale)	40
No. of hosts	2
No. of BNNs (small-scale)	10
No. of BNNs (large-scale)	25
No. of BNN layers	3 to 5
Neurons per layer	8 to 64
Resource profile	Edgecore Wedge100BF-32X
Network Emulator	Mininet
Python version	3.8.10

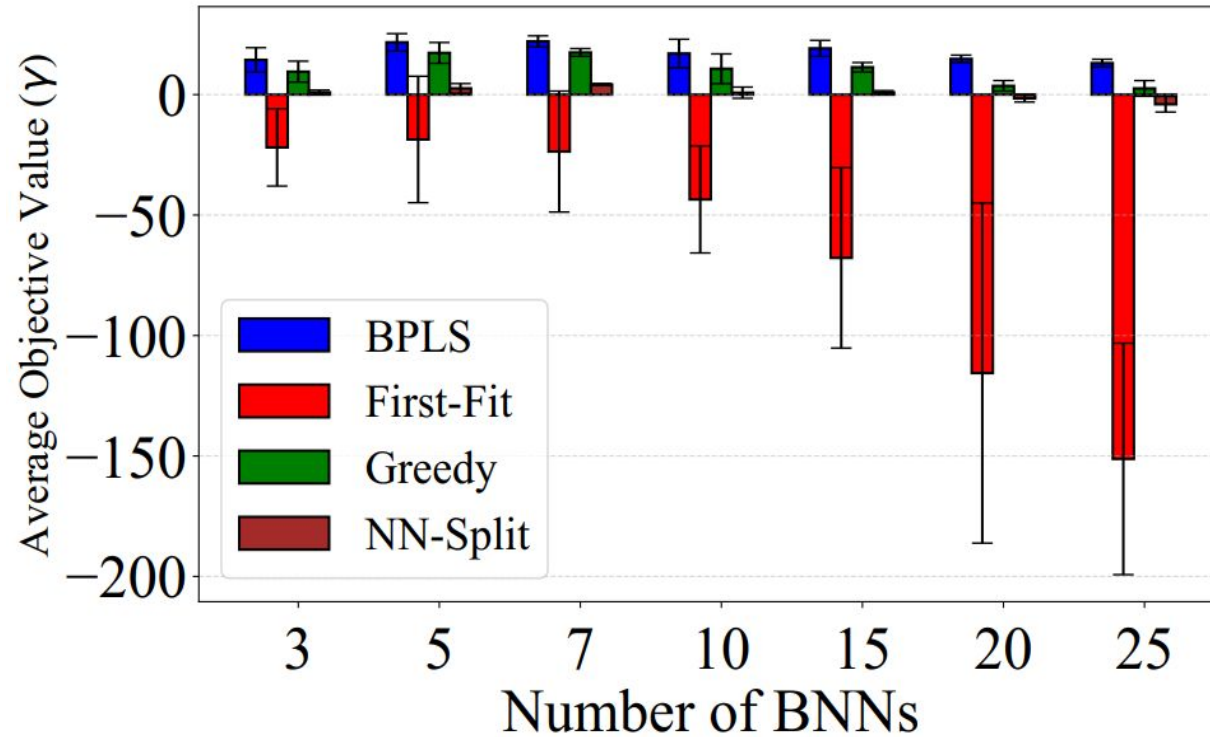
Evaluation

- Impact of the number of BNNs on the objective value
- Baselines
 - First-Fit: place layer on the first switch (ID-sorted) with sufficient capacity.
 - Greedy: Sort layers and switches by popularity and capacity to minimize delay, and free space by removing low-priority duplicate layers when needed to enable new assignment



Evaluation

- Impact of the number of BNNs on the objective value
- Baselines
 - NN-Split: Split BNNs into sub-models and distributes them across multiple switches

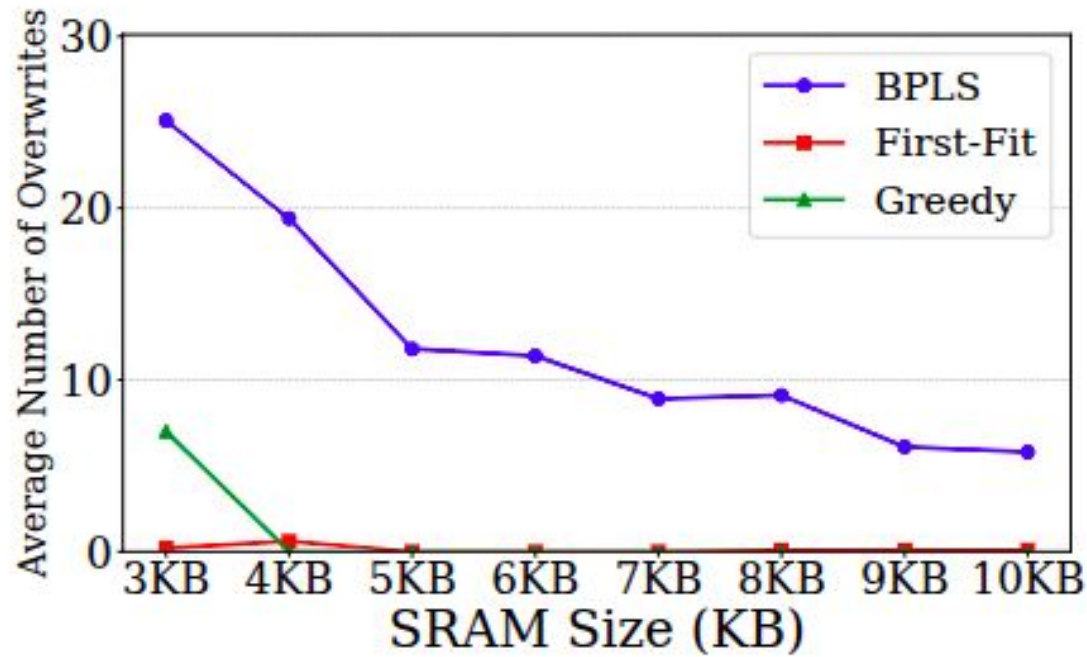


Large-scale

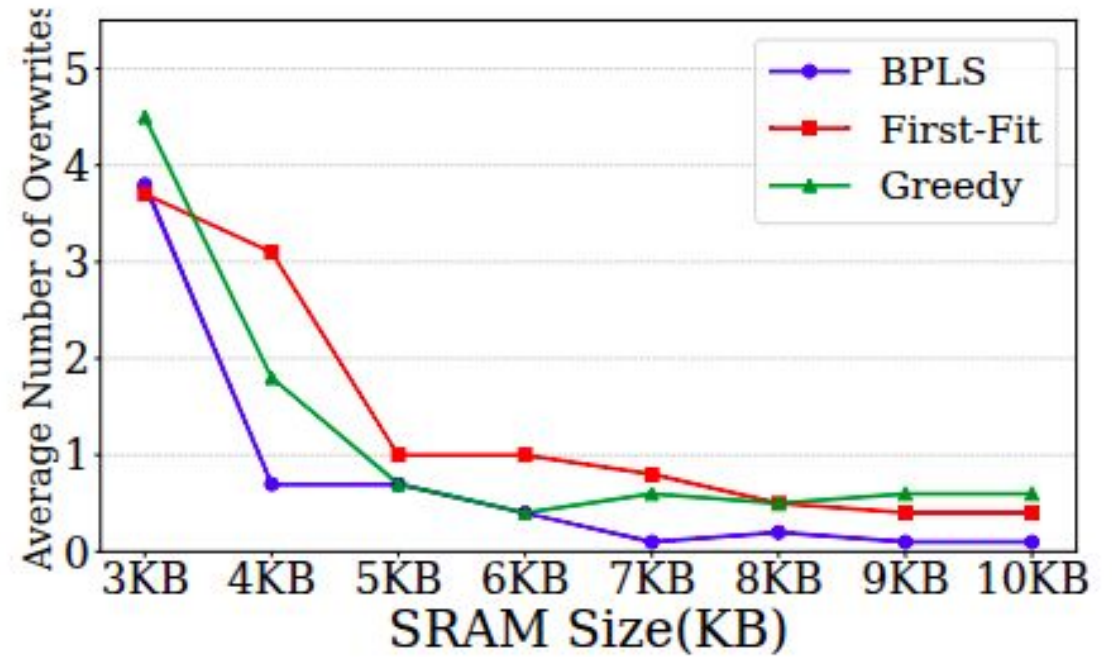
Evaluation

- Impact of memory capacity on the number of overwrites in different networks

BPLS's higher overwrites at low memory are a feature, not a bug.



(a) Large-scale network.



(b) Small-scale network.

Conclusion

- Contribution
 - ILP-based formulation for BNN layer allocation in Programmable switches
 - relaxed-rounding algorithm to optimize resource usage and latency
- Simulations
 - Implemented the solution in P4 and tested it in Mininet
- Future research
 - Hardware deployment
 - Evaluate against boosting-based ML methods.
 - Trying to deploy QoE models on Switches



Thank you!

Any questions?